

<https://www.kaggle.com/code/yunnapudnik/aircraft-bird-s-strikes-analysis>

Vorüberlegungen:

- Steigen oder Fallen die Vogeltreffer über die Jahre?
- In welchen monaten gibt es die Wann passieren die zusammenstöße?
- welche vögel haben am meisten schaden errichtet?

```
In [53]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# CSV-Datei einlesen
df = pd.read_csv("birds_and_planes.csv")
print(len(df.columns))
```

66

```
C:\Users\justi\AppData\Local\Temp\ipykernel_23728\2101440464.py:6: DtypeWarning:
Columns (9,12,15,17) have mixed types. Specify dtype option on import or set low_
memory=False.
    df = pd.read_csv("birds_and_planes.csv")
```

```
In [ ]: # Speichern aller relevanten Daten
incident_year = df["Incident Year"]
incident_phase = df["Flight Phase"]
incident_animal = df["Species Name"]
incident_impact = df["Flight Impact"]
incident_airport = df["Airport"]
incident_month = df["Incident Month"]
```

```
In [59]: # Berechnung des Anstieges von 1990 - 2015
verfügbare_jahre = df['Incident Year'].unique()
print(f"Verfügbare Jahre: {sorted(verfügbare_jahre)}")

if 1990 in verfügbare_jahre and 2015 in verfügbare_jahre:
    birdstrikes_1990 = df[df['Incident Year'] == 1990].shape[0]
    birdstrikes_2015 = df[df['Incident Year'] == 2015].shape[0]

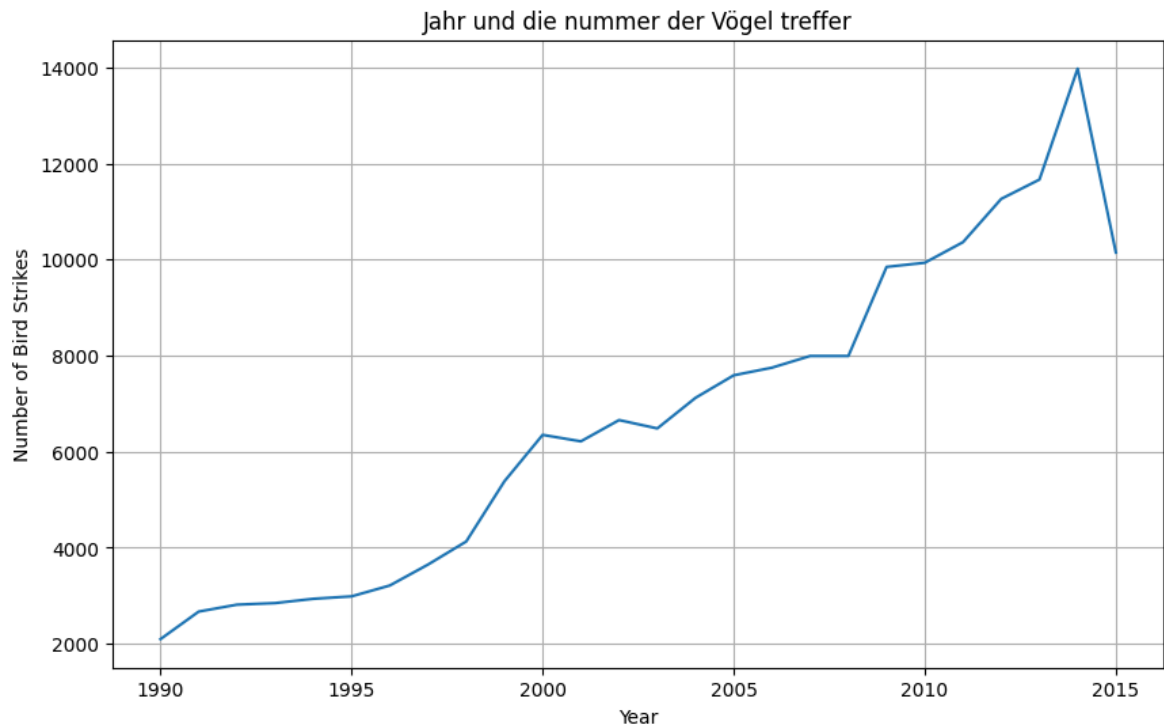
    anstieg_absolut = birdstrikes_2015 - birdstrikes_1990
    anstieg_prozent = ((birdstrikes_2015 - birdstrikes_1990) / birdstrikes_1990)

    print(f"\nAnstieg 1990-2015: +{anstieg_absolut} (+{anstieg_prozent:.1f}%)")
else:
    print("Fehlende Daten für 1990 oder 2015")

# Trend seit 1990
year_counts = incident_year.value_counts().sort_index()
plt.figure(figsize=(10,6))
plt.title("Jahr und die nummer der Vögel treffer")
plt.plot(year_counts.index, year_counts.values)
plt.xlabel("Year")
plt.ylabel("Number of Bird Strikes")
plt.grid()
plt.show()
```

Verfügbare Jahre: [np.int64(1990), np.int64(1991), np.int64(1992), np.int64(1993), np.int64(1994), np.int64(1995), np.int64(1996), np.int64(1997), np.int64(1998), np.int64(1999), np.int64(2000), np.int64(2001), np.int64(2002), np.int64(2003), np.int64(2004), np.int64(2005), np.int64(2006), np.int64(2007), np.int64(2008), np.int64(2009), np.int64(2010), np.int64(2011), np.int64(2012), np.int64(2013), np.int64(2014), np.int64(2015)]

Anstieg 1990-2015: +8049 (+383.5%)



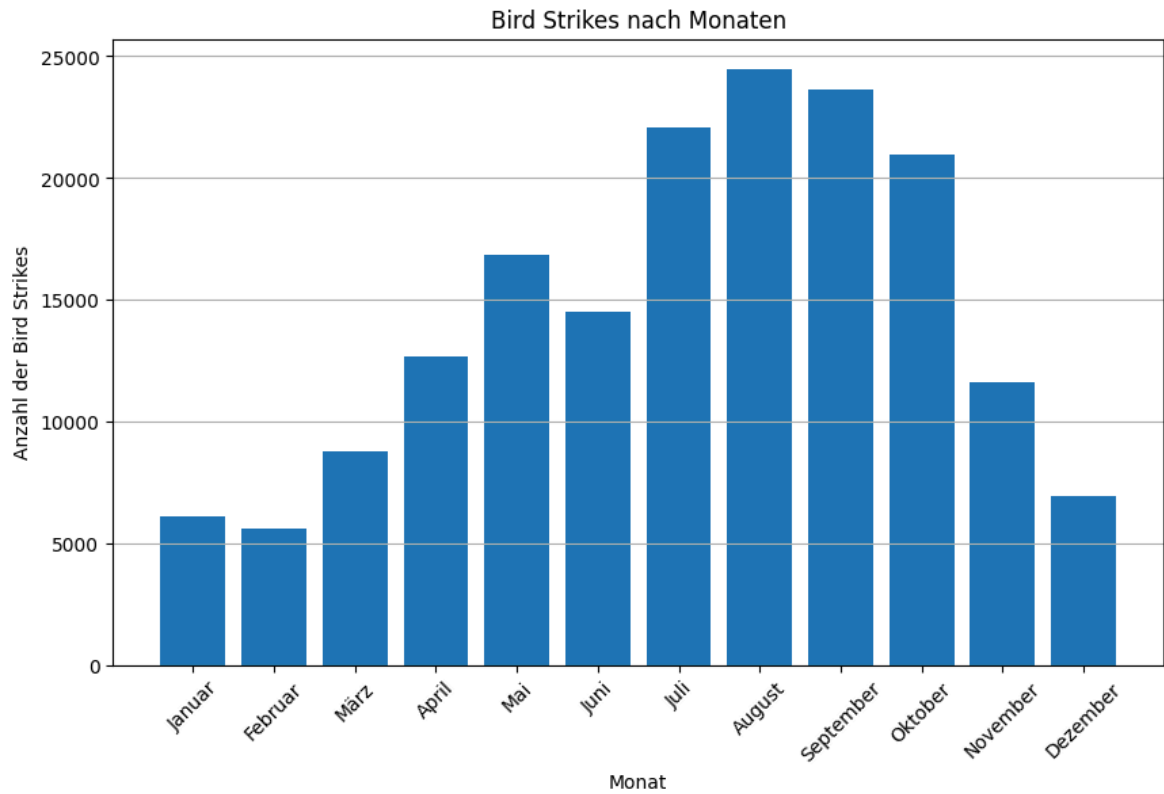
```
In [32]: # Monatsnamen Mapping
monats_namen = {
    1: "Januar", 2: "Februar", 3: "März", 4: "April",
    5: "Mai", 6: "Juni", 7: "Juli", 8: "August",
    9: "September", 10: "Oktober", 11: "November", 12: "Dezember"
}

# Monatliche Verteilung
month_counts = incident_month.value_counts().sort_index()

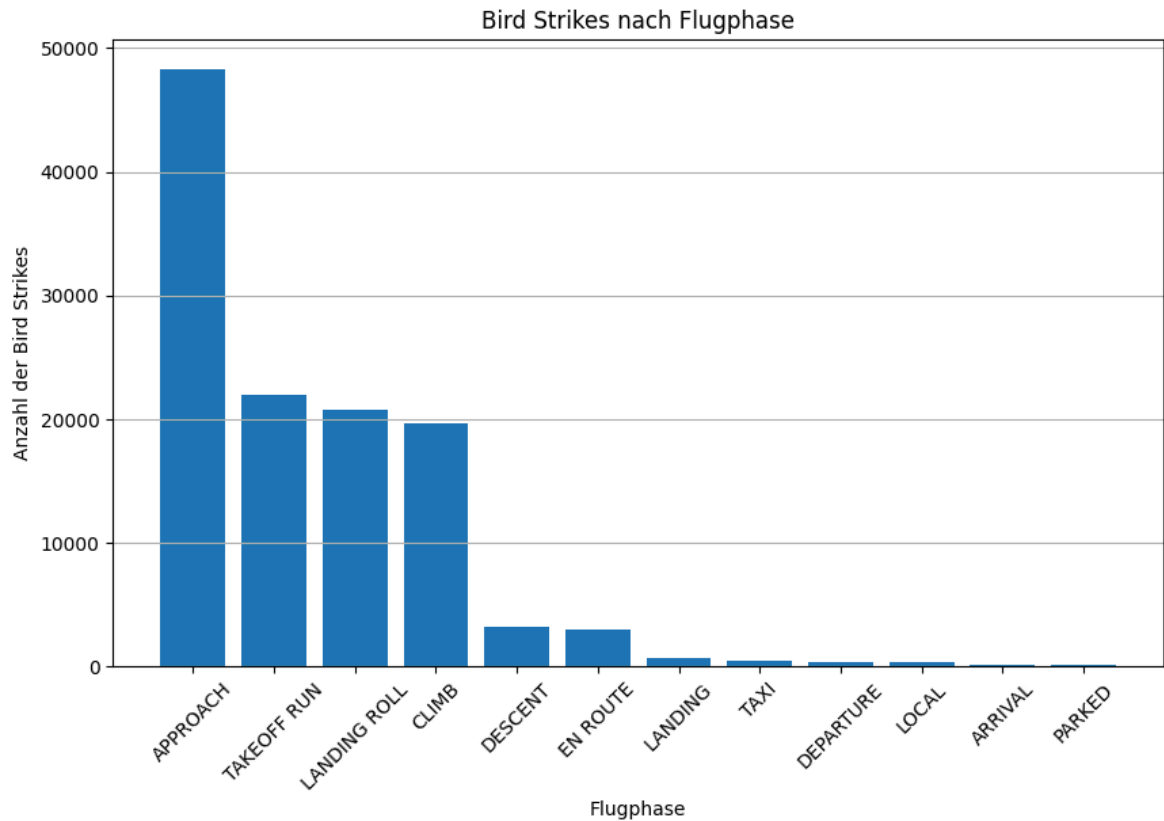
# Erstelle neue Liste mit Monatsnamen
monats_namen_liste = [monats_namen[monat] for monat in month_counts.index]
monats_werte = month_counts.values

# Plot

plt.figure(figsize=(10,6))
plt.title("Bird Strikes nach Monaten")
plt.bar(monats_namen_liste, monats_werte)
plt.xlabel("Monat")
plt.ylabel("Anzahl der Bird Strikes")
plt.xticks(rotation=45)
plt.grid(axis="y")
plt.show()
```

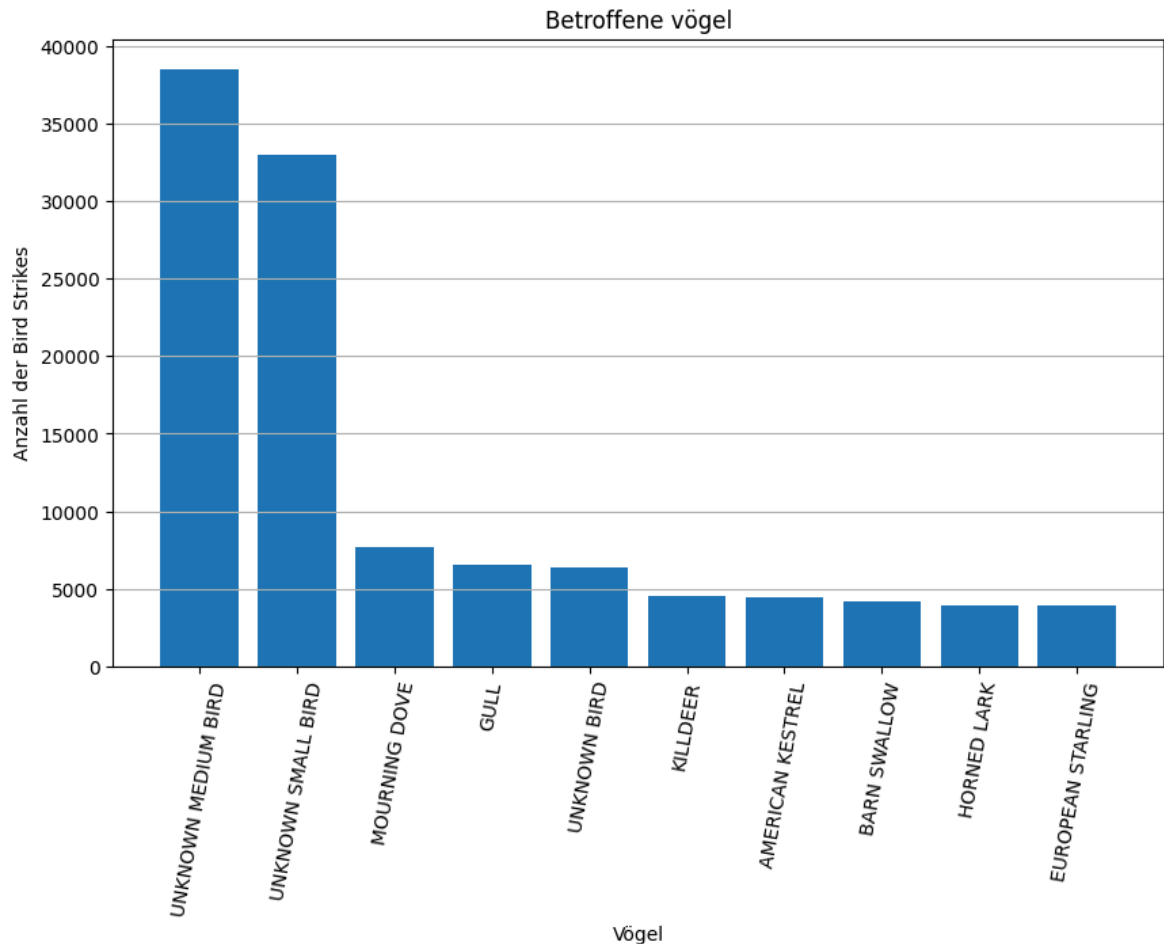


```
In [31]: # Flugphase (Start/Landung)
phase_counts = incident_phase.value_counts()
plt.figure(figsize=(10,6))
plt.title("Bird Strikes nach Flugphase")
plt.bar(phase_counts.index, phase_counts.values)
plt.xlabel("Flugphase")
plt.ylabel("Anzahl der Bird Strikes")
plt.xticks(rotation=45)
plt.grid(axis="y")
plt.show()
print(phase_counts)
```



```
Flight Phase
APPROACH      48275
TAKEOFF RUN   21953
LANDING ROLL  20762
CLIMB         19692
DESCENT       3174
EN ROUTE      2989
LANDING       626
TAXI          397
DEPARTURE     369
LOCAL         316
ARRIVAL       154
PARKED        95
Name: count, dtype: int64
```

```
In [ ]: # Welche der häufigsten vögel sind betroffen?
bird_count = incident_animal.value_counts()
print(len(bird_count)) # Anzahl der Vogelarten
bird_count = incident_animal.value_counts().head(10)
plt.figure(figsize=(10,6))
plt.title("Betroffene vögel")
plt.bar(bird_count.index, bird_count.values)
plt.xlabel("Vögel")
plt.ylabel("Anzahl der Bird Strikes")
plt.xticks(rotation=80)
plt.grid(axis="y")
plt.show()
print(bird_count)
```



```
Species Name
UNKNOWN MEDIUM BIRD    38531
UNKNOWN SMALL BIRD     32981
MOURNING DOVE          7653
GULL                   6580
UNKNOWN BIRD           6400
KILLDEER               4562
AMERICAN KESTREL       4476
BARN SWALLOW           4215
HORNERD LARK           3952
EUROPEAN STARLING      3915
Name: count, dtype: int64
```

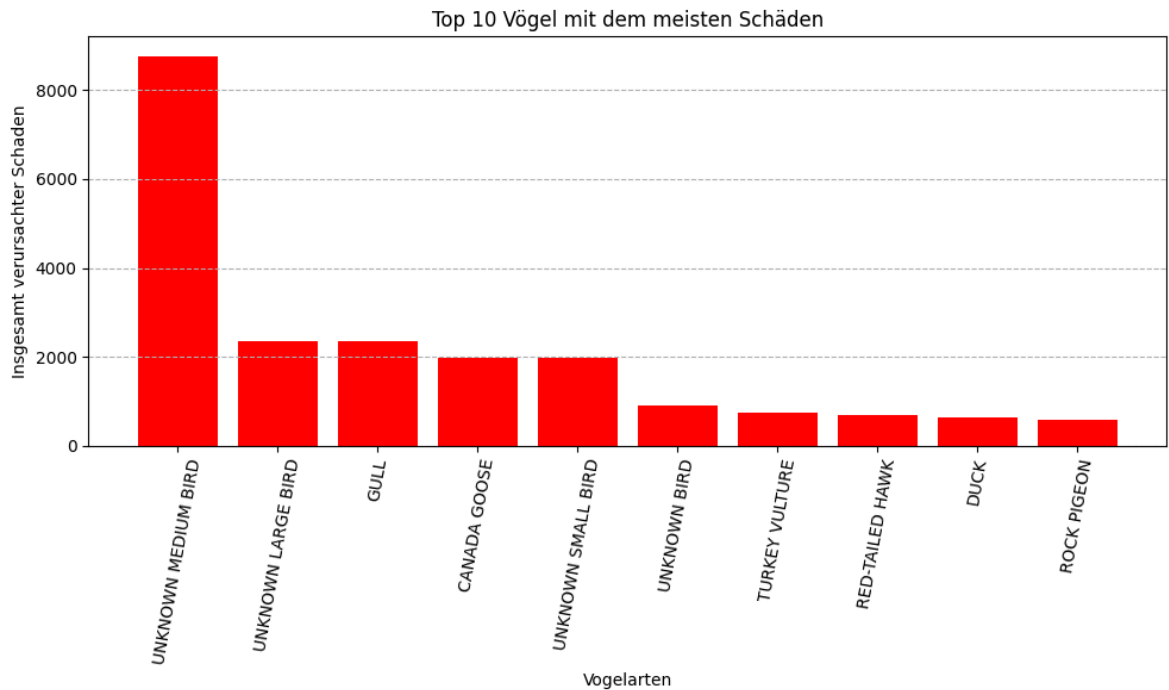
```
In [ ]: # Welche Vögel verursachen den meisten Schaden?
damage_columns = [col for col in df.columns if 'Damage' in col]

# Filtere nur Vögel (keine Rehe/Säugetiere)
voegel_ohne_saeugetiere = df[~df['Species Name'].str.contains('DEER', case=False)]

# Daten Speichern
schaden_pro_vogel = voegel_ohne_saeugetiere.groupby('Species Name')[damage_columns]
gesamt_schaden = schaden_pro_vogel.sum(axis=1) # Summiere alle Schadensarten pro
top_10_voegel = gesamt_schaden.sort_values(ascending=False).head(10) # Sortiere

# Plot erstellen
plt.figure(figsize=(10, 6))
plt.bar(top_10_voegel.index, top_10_voegel.values, color='red')
plt.title("Top 10 Vögel mit dem meisten Schäden")
plt.xlabel("Vogelarten")
plt.ylabel("Insgesamt verursachter Schaden")
plt.xticks(rotation=80)
plt.grid(axis='y', linestyle='--')
```

```
plt.tight_layout()
plt.show()
```

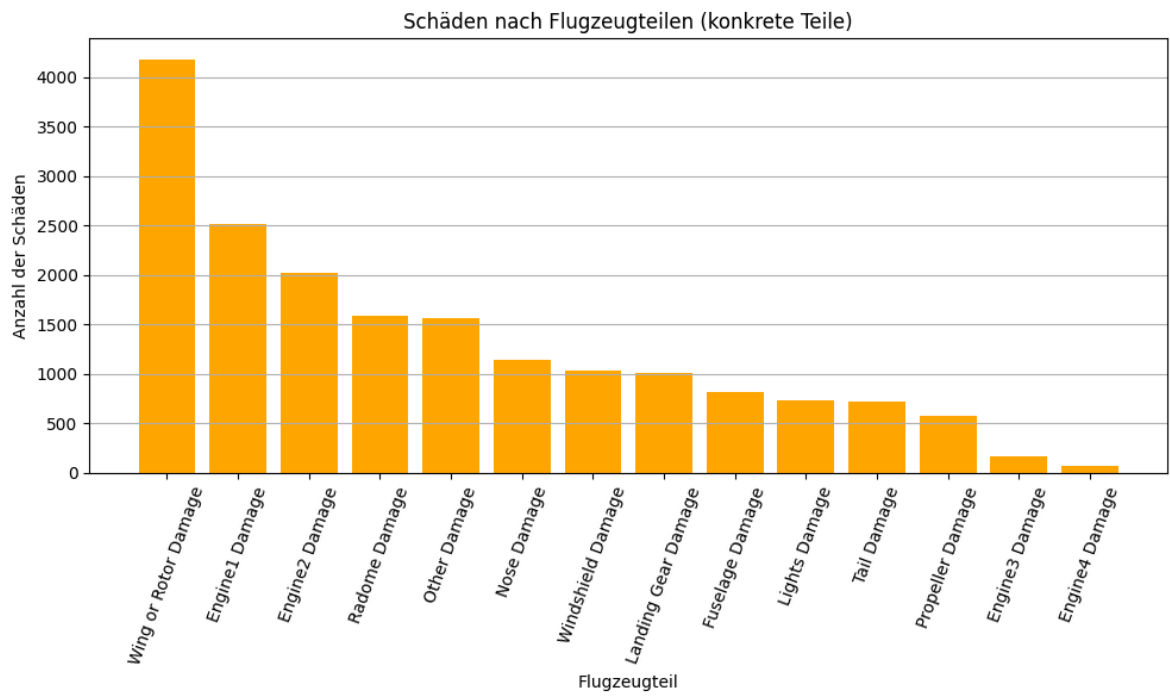


```
In [ ]: # Welcher Teil des Flugzeugs bekommt am meisten Schaden? (ohne Aircraft Damage)
damage_columns = [col for col in df.columns if 'Damage' in col and col != 'Aircraft Damage']

print(len(damage_columns)) # Anzahl möglicher beschädigter Teile

# Summiere Schäden pro Flugzeugteil
schaden_pro_teil = df[damage_columns].sum().sort_values(ascending=False)

plt.figure(figsize=(10, 6))
plt.bar(schaden_pro_teil.index, schaden_pro_teil.values, color='orange')
plt.title("Schäden nach Flugzeugteilen (konkrete Teile)")
plt.xlabel("Flugzeugteil")
plt.ylabel("Anzahl der Schäden")
plt.xticks(rotation=70)
plt.grid(axis="y")
plt.tight_layout()
plt.show()
```



In []: